

LSTM-Based Time-Series Forecasting

Assignment Report

1. Objective

To develop an LSTM (Long Short-Term Memory) based deep learning model for time-series forecasting using historical stock-price data. The model uses previous stock closing prices to predict the next trading day's closing price.

2. Dataset Used

The Apple Inc. (AAPL) historical stock-price dataset is used for this assignment. The data is downloaded programmatically using the yfinance Python library.

Dataset	Apple (AAPL) Historical Stock Prices
Data Source	Yahoo Finance via yfinance
Period	2015-01-01 to 2026-01-01
Target Variable	Closing Price
Sequence Length	60 trading days
Train-Test Split	80% Training / 20% Testing

3. Theory

LSTM is a type of Recurrent Neural Network (RNN) designed to learn patterns from sequential data. Unlike a basic RNN, an LSTM uses memory cells and gates to control the flow of information, allowing it to retain useful information over longer sequences and reduce the vanishing-gradient problem.

Main components of an LSTM cell:

- Forget Gate – determines which information from the previous cell state should be discarded.
- Input Gate – determines which new information should be stored.
- Cell State – carries relevant information through the sequence.
- Output Gate – determines the information passed to the next layer/output.

4. Methodology

1. Download historical AAPL stock-price data.
2. Select the Closing Price as the forecasting target.
3. Remove missing values.
4. Normalize the closing prices using Min-Max Scaling to the range 0–1.
5. Split the dataset into 80% training data and 20% testing data.
6. Create sequences using the previous 60 trading days as input.

7. Reshape the data into the three-dimensional format required by LSTM: samples $\times$ time steps $\times$ features.
8. Build an LSTM neural network with two LSTM layers, Dropout layers, Dense layers, and a single output neuron.
9. Train the model using the Adam optimizer and Mean Squared Error loss.
10. Evaluate predictions using MAE and RMSE.
11. Plot actual versus predicted prices.
12. Use the latest 60 observations to generate a next-trading-day forecast.

5. Model Architecture

	Configuration	Purpose
Input	60 time steps $\times$ 1 feature	Receives previous 60 closing prices
LSTM 1	64 units, return_sequences=True	Learns temporal patterns
Dropout	20%	Reduces overfitting
LSTM 2	64 units	Extracts higher-level sequence patterns
Dropout	20%	Reduces overfitting
Dense	32 units	Learns final nonlinear representation
Output	1 unit	Predicts next closing price

Optimizer: Adam

Loss Function: Mean Squared Error (MSE)

Epochs: Up to 50, with Early Stopping

Batch Size: 32

Validation Split: 10%

6. Important Python Code

```
# Create sequences
time_step = 60

def create_sequences(dataset, time_step):
    X, y = [], []

    for i in range(time_step, len(dataset)):
        X.append(dataset[i-time_step:i, 0])
        y.append(dataset[i, 0])

    return np.array(X), np.array(y)

# Build LSTM model
model = Sequential([
    Input(shape=(X_train.shape[1], 1)),
    LSTM(64, return_sequences=True),
```

```

        Dropout(0.2),
        LSTM(64),
        Dropout(0.2),
        Dense(32),
        Dense(1)
    ])

model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)

# Train
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stopping],
    verbose=1
)

```

7. Evaluation Metrics

Mean Absolute Error (MAE) measures the average absolute difference between the actual and predicted values.

$$\text{MAE} = (1/n) \times \sum |\text{Actual} - \text{Predicted}|$$

Root Mean Squared Error (RMSE) measures the square root of the average squared prediction error and gives greater weight to larger errors.

$$\text{RMSE} = \sqrt{[(1/n) \times \sum (\text{Actual} - \text{Predicted})^2]}$$

8. Results and Observations

After training, the model predicts the closing prices for the test period. The predictions are converted back from normalized values to the original USD price scale using inverse Min-Max scaling.

- The training and validation loss curves can be used to observe the learning behavior of the model.
- The Actual vs Predicted graph provides a visual comparison of model performance.
- MAE and RMSE provide numerical measures of prediction error.
- The trained model can use the latest 60 trading-day sequence to produce a next-day forecast.

9. Advantages of LSTM for Time-Series Forecasting

- Can learn temporal dependencies in sequential data.
- Can retain relevant information across multiple time steps.
- Suitable for nonlinear time-series patterns.
- Can be applied to stock prices, weather measurements, sales, sensor data, and other sequential datasets.

10. Limitations

- Stock prices are affected by many external factors that are not included in this single-variable model.
- Historical price patterns do not guarantee future market behavior.
- The model may not capture sudden events or structural changes in the market.
- The forecast is intended for academic demonstration and should not be treated as financial advice.

11. Conclusion

An LSTM-based time-series forecasting model was successfully developed using historical Apple stock closing prices. The data was cleaned, normalized, and converted into sequences of 60 previous trading days. The LSTM model learned temporal patterns from these sequences and generated predictions for the next day's closing price. The model was evaluated using MAE and RMSE, and actual versus predicted prices were visualized. This experiment demonstrates how LSTM networks can be applied to time-series forecasting problems.